UNIVERSITI TEKNOLOGI MALAYSIA

FACULTY OF COMPUTING

CHAPTER 11 LAB MANUAL

Securing the API with JWT

(Add registration, login, and JWT-protected endpoints)

FIELD	DETAILS
Course	SCSM2223 — Cross-Platform Application Development
Chapter	11 — Securing the API with JWT
Lab time	2 hours
Lab assessment	Lab exercise component
Software	Laragon, Composer, HeidiSQL, Sublime Text / VS Code
Builds on	Ch10_BooksAPI (copy as starting point)

1. Lab Objectives

By the end of this lab you will be able to:

1. Create a users table and store passwords as hashes using password_hash.
2. Install and configure firebase/php-jwt to issue and verify JWTs.
3. Build POST /auth/register and POST /auth/login endpoints.
4. Build a PSR-15 AuthMiddleware that verifies the Authorization: Bearer header.
5. Protect selected /api/books routes so only authenticated callers can write.
6. Add a role-based check that lets only admins delete books.
7. Test the full flow end-to-end with the VS Code REST Client (or Postman).

2. Prerequisites

Verify each item before starting. The fastest way to start is to copy your Chapter 10 project folder to a new folder named ***books-api-jwt*** and work from there.

#	Tool	Verify by running…	Expected
1	Laragon (Full)	Open Laragon, click Start All	Apache + MySQL green
2	PHP 8.1+	php -v	PHP 8.1.x or newer
3	Composer	composer --version	Composer 2.x
4	MySQL access	mysql -u root -e 'SELECT 1'	Returns 1
5	Ch10 project	Copy Ch10_BooksAPI_Solution to a new folder, run composer install	Books API still works

TIP Keep two terminals open in the Laragon Terminal: one running php -S localhost:8000 -t public, one for running composer / mysql.

3. What We Will Build

Starting from the Chapter 10 Books API, we will add:

1. A users table (id, name, email, password_hash, role).
2. POST /auth/register — creates a user with a hashed password (returns 201).
3. POST /auth/login — verifies credentials and returns a signed JWT (returns 200 + token).
4. GET /auth/me — returns the current user (requires JWT).
5. AuthMiddleware — checks Authorization: Bearer header, attaches the decoded payload.
6. Route protection — POST/PUT on /api/books require a token, DELETE requires admin role.

Part A — Database changes

Step 1 — Create the users table

Run this in HeidiSQL (or `mysql -u root`):

```
USE books_api;

DROP TABLE IF EXISTS users;
CREATE TABLE users (
    id            INT AUTO_INCREMENT PRIMARY KEY,
    name          VARCHAR(150) NOT NULL,
    email         VARCHAR(190) NOT NULL UNIQUE,
    password_hash VARCHAR(255) NOT NULL,
    role          ENUM('member','admin') NOT NULL DEFAULT 'member',
    created_at    DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at    DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP
                               ON UPDATE CURRENT_TIMESTAMP
) ENGINE=InnoDB;
```

Step 2 — Seed two demo users

Generate a hash from the Laragon Terminal:

```
php -r "echo password_hash('password', PASSWORD_DEFAULT);"
```

Copy the output (something like $2y$12$...) and use it in:

```
INSERT INTO users (name, email, password_hash, role) VALUES
  ('Demo Admin',  'admin@books.test',  '<paste hash here>', 'admin'),
  ('Demo Member', 'member@books.test', '<paste hash here>', 'member');
```

CHECKPOINT SELECT id, name, email, role FROM users; should show two rows.

Part B — Install firebase/php-jwt

Step 3 — Add the package

```
composer require firebase/php-jwt
```

Step 4 — Add JWT settings to .env

```
JWT_SECRET=change-me-to-a-long-random-string-please-do-not-commit
JWT_TTL=3600
JWT_ISSUER=books-api
```

Generate a real secret:

```
php -r "echo bin2hex(random_bytes(32));"
```

WARNING Anyone with JWT_SECRET can issue valid tokens. Never commit .env or share the secret.

Part C — JwtService and UserRepository

Step 5 — src/Auth/JwtService.php

```php
<?php
namespace App\Auth;
use Firebase\JWT\JWT;
use Firebase\JWT\Key;

final class JwtService {
    private string $secret;
    private string $algo = 'HS256';
    private int    $ttl;
    private string $issuer;

    public function __construct() {
        $this->secret = $_ENV['JWT_SECRET'];
        $this->ttl    = (int)($_ENV['JWT_TTL'] ?? 3600);
        $this->issuer = $_ENV['JWT_ISSUER'] ?? 'books-api';
    }
    public function issue(int $userId, array $extra = []): string {
        $now = time();
        $payload = array_merge([
            'iss' => $this->issuer, 'sub' => $userId,
            'iat' => $now, 'exp' => $now + $this->ttl,
        ], $extra);
        return JWT::encode($payload, $this->secret, $this->algo);
    }
    public function verify(string $token): array {
        return (array)JWT::decode($token, new Key($this->secret, $this->algo));
    }
    public function ttl(): int { return $this->ttl; }
}
```

Step 6 — src/Repositories/UserRepository.php

```php
<?php
namespace App\Repositories;
use PDO;

final class UserRepository {
    public function __construct(private PDO $pdo) {}

    public function findByEmail(string $email): ?array {
        $stmt = $this->pdo->prepare(
            'SELECT id, name, email, password_hash, role FROM users WHERE email = :e'
        );
        $stmt->execute([':e' => mb_strtolower(trim($email))]);
        $row = $stmt->fetch();
        return $row === false ? null : $row;
    }
    public function findById(int $id): ?array {
        $stmt = $this->pdo->prepare('SELECT id, name, email, role FROM users WHERE id = :id');
        $stmt->execute([':id' => $id]);
```

```
        $row = $stmt->fetch();
        return $row === false ? null : $row;
    }
    public function create(string $n, string $e, string $hash, string $role='member'): int {
        $this->pdo->prepare(
            'INSERT INTO users (name,email,password_hash,role) VALUES (:n,:e,:h,:r)'
        )->execute([':n'=>trim($n), ':e'=>mb_strtolower(trim($e)), ':h'=>$hash, ':r'=>$role]);
        return (int)$this->pdo->lastInsertId();
    }
    public function emailExists(string $e): bool {
        $stmt = $this->pdo->prepare('SELECT 1 FROM users WHERE email = :e');
        $stmt->execute([':e' => mb_strtolower(trim($e))]);
        return (bool)$stmt->fetchColumn();
    }
}
```

Part D — AuthController

Step 7 — src/Controllers/AuthController.php (register + login)

```
<?php
namespace App\Controllers;
use App\Auth\JwtService;
use App\Repositories\UserRepository;
use Psr\Http\Message\ResponseInterface as Response;
use Psr\Http\Message\ServerRequestInterface as Request;

final class AuthController {
    public function __construct(
        private UserRepository $users,
        private JwtService     $jwt,
    ) {}

    public function register(Request $r, Response $s): Response {
        $b = (array)$r->getParsedBody();
        $errors = [];
        if (empty($b['name'])  || mb_strlen($b['name']) < 2) $errors['name']  = 'min 2 chars';
        if (empty($b['email']) || !filter_var($b['email'], FILTER_VALIDATE_EMAIL))
            $errors['email'] = 'invalid email';
        if (empty($b['password']) || mb_strlen($b['password']) < 6)
            $errors['password'] = 'min 6 chars';
        if ($errors) return $this->json($s, ['errors'=>$errors], 400);
        if ($this->users->emailExists($b['email']))
            return $this->json($s, ['error'=>'Email already registered'], 409);
        $id = $this->users->create($b['name'], $b['email'],
                  password_hash($b['password'], PASSWORD_DEFAULT));
        return $this->json($s, ['message'=>'Registered',
                  'user'=>$this->users->findById($id)], 201);
    }

    public function login(Request $r, Response $s): Response {
        $b = (array)$r->getParsedBody();
        $u = $this->users->findByEmail($b['email'] ?? '');
        if (!$u || !password_verify($b['password'] ?? '', $u['password_hash']))
```

```
            return $this->json($s, ['error'=>'Invalid credentials'], 401);
        $token = $this->jwt->issue((int)$u['id'], ['role'=>$u['role'], 'email'=>$u['email']]);
        return $this->json($s, [
            'token_type'=>'Bearer','expires_in'=>$this->jwt->ttl(),
            'access_token'=>$token,
        ]);
    }

    public function me(Request $r, Response $s): Response {
        $auth = (array)$r->getAttribute('auth', []);
        $u = $this->users->findById((int)($auth['sub'] ?? 0));
        return $u ? $this->json($s, $u)
                  : $this->json($s, ['error'=>'Not found'], 404);
    }
    private function json(Response $s, $d, int $c=200): Response {
        $s->getBody()->write(json_encode($d, JSON_PRETTY_PRINT));
        return $s->withHeader('Content-Type','application/json')->withStatus($c);
    }
}
```

Part E — AuthMiddleware

Step 8 — src/Middleware/AuthMiddleware.php

```
<?php
namespace App\Middleware;
use App\Auth\JwtService;
use Psr\Http\Message\ResponseInterface;
use Psr\Http\Message\ServerRequestInterface;
use Psr\Http\Server\MiddlewareInterface;
use Psr\Http\Server\RequestHandlerInterface;
use Slim\Psr7\Response as SlimResponse;

final class AuthMiddleware implements MiddlewareInterface {
    public function __construct(private JwtService $jwt) {}

    public function process(ServerRequestInterface $req, RequestHandlerInterface $h):
ResponseInterface {
        $hdr = $req->getHeaderLine('Authorization');
        if (!preg_match('/^Bearer\s+(.+)$/i', $hdr, $m)) {
            return $this->fail('Missing or malformed token');
        }
        try {
            $payload = $this->jwt->verify($m[1]);
        } catch (\Throwable $e) {
            error_log('[Auth] '.$e->getMessage());
            return $this->fail('Invalid or expired token');
        }
        $req = $req->withAttribute('auth', $payload);
        return $h->handle($req);
    }
    private function fail(string $msg): ResponseInterface {
        $r = new SlimResponse(401);
        $r->getBody()->write(json_encode(['error'=>$msg]));
```

```
        return $r->withHeader('Content-Type','application/json')
                 ->withHeader('WWW-Authenticate','Bearer');
    }
}
```

Part F — Wire it all up in routes.php

Step 9 — Build dependencies and apply middleware

```
<?php
use App\Auth\JwtService;
use App\Controllers\AuthController;
use App\Controllers\BookController;
use App\Database;
use App\Middleware\AuthMiddleware;
use App\Repositories\BookRepository;
use App\Repositories\UserRepository;
use Slim\App;

return function (App $app): void {
    $pdo  = Database::get();
    $jwt  = new JwtService();
    $auth = new AuthMiddleware($jwt);

    $bookCtrl = new BookController(new BookRepository($pdo));
    $authCtrl = new AuthController(new UserRepository($pdo), $jwt);

    // Public
    $app->post('/auth/register', [$authCtrl, 'register']);
    $app->post('/auth/login',    [$authCtrl, 'login']);
    $app->get ('/api/books',     [$bookCtrl, 'index']);
    $app->get ('/api/books/{id}',[$bookCtrl, 'show']);

    // Protected
    $app->get('/auth/me', [$authCtrl, 'me'])->add($auth);
    $app->group('/api/books', function ($g) use ($bookCtrl) {
        $g->post  ('',      [$bookCtrl, 'create']);
        $g->put   ('/{id}', [$bookCtrl, 'update']);
        $g->delete('/{id}', [$bookCtrl, 'delete']);
    })->add($auth);
};
```

Step 10 — Add a role check inside delete()

Open BookController::delete() and add this guard at the top:

```
$auth = (array)$req->getAttribute('auth', []);
if (($auth['role'] ?? 'member') !== 'admin') {
    return $this->json($res, ['error' => 'Admins only'], 403);
}
```

Part G — End-to-End Tests

Restart the server: php -S localhost:8000 -t public. Run each test in order.

Test 1 — Register

```
POST http://localhost:8000/auth/register
Content-Type: application/json

{ "name":"Hilang", "email":"hilang@example.com", "password":"password123" }

Expected: 201
```

Test 2 — Login (member)

```
POST http://localhost:8000/auth/login
Content-Type: application/json

{ "email":"member@books.test", "password":"password" }

Expected: 200 — copy access_token from the response
```

Test 3 — Use the token

```
GET http://localhost:8000/auth/me
Authorization: Bearer <paste token here>

Expected: 200, your user object
```

Test 4 — Create a book (authenticated)

```
POST http://localhost:8000/api/books
Authorization: Bearer <token>
Content-Type: application/json

{ "title":"Refactoring", "author":"Martin Fowler", "year":2018 }

Expected: 201
```

Test 5 — Member tries to delete (expect 403)

```
DELETE http://localhost:8000/api/books/1
Authorization: Bearer <member token>

Expected: 403 "Admins only"
```

Test 6 — Login as admin and delete (expect 200)

```
POST http://localhost:8000/auth/login          → admin@books.test / password
DELETE http://localhost:8000/api/books/1 with the new token → 200
```

Test 7 — No token (expect 401)

```
POST http://localhost:8000/api/books with NO Authorization header
Expected: 401 "Missing or malformed token"
```

Test 8 — Tampered token (expect 401)

```
GET http://localhost:8000/auth/me
Authorization: Bearer eyJhbGciOiJIUzI1NiJ9.tampered.signature

Expected: 401 "Invalid or expired token"
```

LESSON The middleware never trusts the payload — it always re-verifies the signature with your server's secret. Without that secret, no attacker can forge a valid JWT.

Part H — Self-Check & Extension Tasks

7. Confirm the seeded admin can DELETE; a member gets 403.
8. Confirm POST/PUT without a token return 401 with WWW-Authenticate: Bearer.
9. Decode the JWT at jwt.io and verify the payload contains sub, role, iat, exp.
10. Set JWT_TTL=10 in .env, login, wait 11 seconds, hit /auth/me — observe 401.
11. BONUS — add a refresh-token table and POST /auth/refresh that issues a new access token.
12. BONUS — add a per-user 'created_by' column to books and stamp it on POST /api/books using $req->getAttribute('auth')['sub'].
13. BONUS — write a tiny Vue page that logs in, stores the token in Pinia, and lists books authenticated.

Appendix — Troubleshooting

Symptom	Likely cause	Fix
JWT_SECRET is missing or still set to the placeholder…	You forgot to set JWT_SECRET	Generate a real value with `php -r "echo bin2hex(random_bytes(32));"` and put it in .env.
Always 401, even with the correct token	The middleware uses a different secret than the issuer	Make sure JwtService is constructed once and reads the same .env.
password_verify() always returns false	The hash column is too short / truncated	VARCHAR(255) for password_hash.
TypeError on JWT::decode	Old library version	Run `composer update firebase/php-jwt` (you need ^6.x).
Token works but role check always 403	Role wasn't added to the payload	Make sure $jwt->issue() includes 'role' => $user['role'].
CORS preflight fails for /auth/login	Authorization header blocked	Cors middleware must list 'Authorization' under Access-Control-Allow-Headers.

End of Lab Manual.